SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Programme: **B.E / B.Tech**

Course Code: **CSA3714**

Course Name: **SOFTWARE TESTING**

Duration: **1 Week**

Submission Date: 4-11-2026

Date of Issue: 2-11-2026 Max Marks: **50 Marks**

ASSIGNMENT – 1

CO	Course Outcome Statement
CO1	Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Scenario — Hospital Outpatient Appointment Application (MediQueue)

A multi-speciality hospital launched MediQueue, a mobile and web application that allows patients to book outpatient appointments, pay consultation fees, receive token numbers and view prescriptions. About 6,000 patients use the application every day.

The application was built in eight months by a team of five developers. There was no separate tester. Developers "tested" their own code just before release, and the release was approved when the demo to the hospital management showed no errors.

Within three months of release, the hospital received many complaints. The management has asked you, as a newly appointed Quality Engineer, to investigate why so many defects reached patients and to recommend how quality should be built into the process.

Incidents reported after release:

ID	Incident
I1	Two patients receive the same token number for the same doctor when they book at the same second.
I2	Appointments for Sunday are accepted although the outpatient department is closed on Sundays; this rule was never written in the requirements.
I3	Consultation fee shown as ₹500 on the booking page but ₹550 is charged; the GST rule was misunderstood.
I4	Application crashes when a patient name has more than 40 characters.
I5	Senior-citizen priority is not applied for patients who turn 60 on the day of the appointment.
I6	Prescription PDF shows the previous patient's details when two doctors save at the same time.
I7	The user manual tells patients to cancel through "My Visits", but this menu does not exist.
I8	A test case for refund was marked "Pass" because the tester compared the output with a wrong expected value.
I9	SMS reminders are not sent when the hospital's SMS provider changes its API key.
I10	The doctor's schedule screen takes 25 seconds to load on the hospital's Wi-Fi network.

Defect data for the last release (100 defects):

Phase	Defects introduced	Defects detected	Cost to fix per defect (₹)	Remarks
Requirements	**30**	**2**	**500**	**No formal review**
Design	**22**	**3**	**1,500**	**No formal review**
Coding	**38**	**15**	**3,000**	**Developer self-check only**
Testing	**5**	**20**	**8,000**	**Demo-based testing**
Documentation	**5**	**0**	**–**	**–**
Production (after release)	**–**	**60**	**40,000**	**Reported by patients**

Design Challenge:

The hospital now plans to release a new version every two weeks and to connect MediQueue to the national digital health ID system.

Re-evaluate your recommended quality process and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate Testing as an Engineering Activity 5 Marks

CO1 | Bloom's Level: K5 – Evaluate

Investigate whether the way MediQueue was tested can be called an engineering activity.

Did the MediQueue team engineer quality into the product, or did they only hope for it?

Discuss:

- **The characteristics of testing as an engineering activity (planning, measurement, standards, repeatability)**
- **Which of these characteristics were missing in the MediQueue project, with evidence from the case**
- **How the absence of each characteristic contributed to the incidents reported**
- **What an engineered testing approach would have looked like for this project**

You are expected to judge the project against engineering practice, not merely define the term.

Evidence Expected

A reasoned evaluation of the MediQueue testing practice against the characteristics of an engineering activity, supported by evidence from the case.

TASK 2 – Evaluate the Role of Process in Software Quality 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

The hospital management believes that hiring "better developers" will solve the quality problem.

Is quality mainly a result of good people, or of a good process?

Discuss:

- **The role of a defined process in achieving software quality**
- **Whether better developers alone would have prevented the incidents**
- **How a process maturity view (for example, TMM levels) would place the current team**

Your position must be defended with evidence from the case.

Evidence Expected

A justified position on the role of process in quality, supported by the case data.

TASK 3 – Evaluate Testing as a Process 6 Marks

CO1 | Bloom's Level: K5 – Evaluate

In MediQueue, testing was a single activity done just before release.

What would change in the defect data if testing had been treated as a process running through the whole life cycle?

Discuss:

- **Testing as a process with planning, design, execution, evaluation and reporting activities**
- **The relationship between verification and validation activities and each development phase**
- **The phase in which each type of MediQueue defect should have been detected**
- **Using the defect data table, the cost the hospital paid because defects were found late**
- **The likely saving if requirements and design reviews had been conducted**
- **The limits of your estimate and the assumptions it depends on**

You must show your calculations and interpret them. Stating that "early testing is cheaper" will not receive full credit.

Evidence Expected

A process-based evaluation of testing in MediQueue, supported by cost calculations from the defect data and a reasoned estimate of savings.

TASK 4 – Apply and Critique Basic Definitions 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

The team uses the words "bug", "error" and "failure" interchangeably in its reports.

Does careless terminology actually harm quality, or is it only a matter of words?

Discuss:

- **The meaning of error, fault (defect), failure, test case, test oracle and test bed in the MediQueue context**
- **For incidents I3, I5 and I6, the error, the fault and the failure**
- **Why the confusion in terminology made incident I8 possible**
- **Whether the team needs a common glossary, and what it must contain**

Definitions copied from a textbook will not receive full credit. Each must be applied to MediQueue.

Evidence Expected

Correct application of basic testing definitions to the MediQueue incidents and a justified judgement on the effect of terminology on quality.

TASK 5 – Evaluate the Application of Testing Principles 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

The developers claim that "we tested everything and found no errors, so the software was correct".

Which testing principles prove this claim wrong?

Discuss:

- **At least four software testing principles that apply to the MediQueue situation**
- **For each principle, the incident or decision in the case that violates or confirms it**
- **Which principle, if followed, would have had the largest effect on quality, and why**

Do not simply list principles. You are expected to relate each principle to evidence.

Evidence Expected

A justified evaluation of the developers' claim using testing principles linked to evidence from the case.

TASK 6 – Evaluate the Tester's Role 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

MediQueue had no independent tester.

Would an independent tester have changed the outcome, or would the same defects still have reached patients?

Discuss:

- **The role and responsibilities of a tester across the life cycle**
- **The advantages and limitations of developers testing their own code**

- **Which incidents a tester would most likely have caught, and at which stage**
- **The qualities a tester for a healthcare application must have**

Your answer must take a position and defend it.

Evidence Expected

A defended judgement on the value of an independent tester for MediQueue, linked to specific incidents.

TASK 7 – Trace the Origins of Defects 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Using the incidents reported after release, investigate where each defect originated.

Where in the life cycle are MediQueue's defects really coming from?

Discuss:

- **The origin of each incident: requirements, design, code, testing, documentation or environment**
- **The evidence in the case that supports each origin you assign**
- **Incidents that may have more than one origin, and how you decided**
- **What the overall pattern of origins tells the hospital about where to invest effort**

Where an incident could belong to more than one origin, argue for your choice.

Evidence Expected

A justified origin for each incident and an interpretation of the overall pattern of defect origins.

TASK 8 – Evaluate Defect Classes and Severity 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

A defect classification scheme helps the team understand the nature of its defects.

Which classes of defects are most harmful to patients, and do the team's priorities reflect that?

Discuss:

- **A suitable defect classification (requirements/specification, design, coding, testing defects and their sub-classes)**
- **The class, severity and priority of each incident, with reasons**
- **Why severity and priority may differ for the same defect (for example, I7 and I1)**
- **Which defect class the team should target first, and why**

Simply placing defects in a table is not sufficient. You are expected to justify each classification.

Evidence Expected

A justified classification of all ten incidents by class, severity and priority, with a recommendation on which class to target first.

TASK 9 – Design and Appraise a Defect Repository 3 Marks

CO1 | Bloom's Level: K5 – Evaluate

The hospital currently records complaints in e-mails and a spreadsheet.

What information must a defect repository hold so that the team can learn from its defects and stop repeating them?

Discuss:

- **The fields a defect repository for MediQueue must contain, and why each is needed**

- **How the repository supports defect analysis, process improvement and prevention**
- **The weaknesses of the current e-mail and spreadsheet approach**

You are expected to justify your repository design, not just list fields.

Evidence Expected

A defect repository design for MediQueue with a justification for each field and an evaluation of its benefits.

TASK 10 – Evaluate Defect Examples and Prevention Strategies 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Select three incidents that represent different defect classes.

For each selected defect, what single change to the process would have prevented it?

Discuss:

- **The root cause of each selected defect**
- **A prevention strategy for each (for example, requirement reviews, design inspection, coding standards, test oracles)**
- **The cost and effort of each strategy compared with its benefit**
- **Which strategy gives the best value for the hospital**

Your recommendation must be supported by evidence from the case and the defect data.

Evidence Expected

A root-cause analysis of three defects with justified, cost-aware prevention strategies.

TASK 11 – Recommend and Defend a Quality Improvement Plan 4 Marks

CO1 | Bloom's Level: K5 – Evaluate

Based on your analysis in Tasks 1–10, recommend a quality improvement plan for MediQueue.

Discuss:

- **The process changes you recommend and their order of priority**
- **The role of testers, reviews and the defect repository in your plan**
- **The expected effect on the number and cost of defects**
- **Trade-offs, risks and limitations of your plan**
- **Whether the plan remains practical for a small team of five developers**

Your recommendation must follow from your own analysis. A generic list of best practices is not sufficient.

Evidence Expected

A technically justified quality improvement plan supported by the evidence developed in earlier tasks.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Quality Process 8 Marks

CO1 | Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital now wants to release a new version of MediQueue every two weeks instead of every six months.

At the same time, MediQueue must connect to the national digital health ID system, which is maintained by an external agency.

Patient health records will now be shared between hospitals.

Re-evaluate your original solution in response to these changed conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed from the original situation**
- **Which assumptions in your quality improvement plan are affected**
- **Why frequent releases change the role of testing and the tester**
- **Why sharing health records raises the severity of some defect classes**

Evidence Expected

A clear analysis of how the changed conditions affect the original quality improvement plan.

B. Evaluation of the Original Plan – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Evaluate whether your recommended plan remains appropriate.

Discuss:

- **Whether the planned reviews and test activities fit into a two-week cycle**

- **Which new defect origins are introduced by the external health ID system**
- **Whether your defect repository is still sufficient**

Evidence Expected

A technically supported evaluation of the suitability of the original plan under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the quality process.

If you modify or redesign it:

- **Describe the revised process and the tester's revised role**
- **Explain which testing principles now become more important**
- **Explain how defects from the external system will be classified and tracked**

If you retain the original plan, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate response to the changed requirement with clear quality-engineering reasoning.

D. Evidence-Based Defence – 2 Marks

CO1 | Bloom's Level: K5 – Evaluate

Defend your final decision using evidence.

Use:

- **The defect data and cost figures**
- **Defect origins and classes**

- **Release frequency**
- **Patient-safety and data-privacy risk**
- **Team size and capacity**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final decision integrating defect evidence, process reasoning and the changed constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Evaluate Testing as an Engineering Activity	**5**
Task 2 – Evaluate the Role of Process in Software Quality	**2**
Task 3 – Evaluate Testing as a Process	**6**
Task 4 – Apply and Critique Basic Definitions	**4**
Task 5 – Evaluate the Application of Testing Principles	**3**
Task 6 – Evaluate the Tester's Role	**3**
Task 7 – Trace the Origins of Defects	**4**

Task 8 – Evaluate Defect Classes and Severity	**4**
Task 9 – Design and Appraise a Defect Repository	**3**
Task 10 – Evaluate Defect Examples and Prevention Strategies	**4**
Task 11 – Recommend and Defend a Quality Improvement Plan	**4**
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

TASK 1 – Evaluate Testing as an Engineering Activity

Answer

Software testing as an engineering activity means following a systematic and planned approach to identify defects and ensure software quality. It involves planning, measurement, standards and repeatability.

In MediQueue, testing was not performed as a proper engineering activity. Developers tested their own code just before release, and the application was approved based only on a demonstration. Therefore, the team hoped for quality instead of engineering it into the product.

1. Characteristics of Testing as an Engineering Activity

1. **Planning:** Testing activities, schedules, resources and test cases should be planned before development. MediQueue had no separate testing plan or tester.
2. **Measurement:** Testing should use measurable results such as defect count, test coverage and performance. MediQueue was approved through a demonstration without proper measurements.
3. **Standards:** Testing should follow defined procedures, coding standards and quality guidelines. MediQueue had no formal requirement reviews or testing standards.
4. **Repeatability:** Tests should be documented so that the same tests can be executed again and produce consistent results. MediQueue did not maintain proper test cases, which resulted in a refund test being incorrectly marked as passed (I8).

2. Missing Characteristics and Their Impact

Missing characteristic	Evidence from MediQueue	Result
Planning	No separate tester or testing plan	Duplicate tokens and booking errors
Measurement	Only demo-based approval	Slow schedule screen was not identified
Standards	No formal requirement reviews	Sunday appointments were accepted
Repeatability	Wrong expected value in refund test	Incorrect test result

3. Engineered Testing Approach

The hospital should introduce:

- Requirement reviews before development to identify missing rules.
- Design reviews to identify duplicate token and patient-data problems.
- Unit, integration, system and acceptance testing.
- Performance and security testing before release.
- Documented test cases with correct expected results.
- Regular defect tracking and reporting.

Conclusion

The MediQueue team did not engineer quality into the product because testing was informal, unplanned and performed only before release. An independent testing process, proper reviews, measurable quality standards and repeatable test cases would have detected many defects earlier and reduced the cost of fixing them.

TASK 2 – Evaluate the Role of Process in Software Quality

Answer

Software quality depends on both skilled developers and a well-defined development and testing process. Good developers alone cannot guarantee a defect-free application.

1. Role of Process in Software Quality

A defined process provides proper planning, requirement analysis, design reviews, coding standards, testing and defect tracking. It helps identify defects early and improves reliability.

2. Can Better Developers Alone Prevent Defects?

No. Even experienced developers can make mistakes.

For example:

- I1: Duplicate tokens occurred due to improper handling of simultaneous bookings.
- I2: Sunday bookings were accepted because the requirement was missing.

- I6: Patient details were mixed because of a data-handling problem.
- I8: A wrong expected value caused an incorrect test result.

These problems cannot be solved only by hiring better developers. Proper reviews and testing procedures are also necessary.

3. Test Maturity Model (TMM)

The current MediQueue team can be placed at **Level 1 – Initial** because testing was informal and depended on individual developers.

The five maturity levels are:

1. Initial – Testing is informal.
2. Definition – Testing processes and policies are defined.
3. Integration – Testing is integrated into the development life cycle.
4. Management and Measurement – Testing activities are measured and controlled.
5. Optimization – Continuous improvement and defect prevention are followed.

Conclusion

Software quality is the responsibility of both people and processes. MediQueue should introduce a defined testing process and gradually improve its maturity to achieve better software quality.

TASK 3 – Evaluate Testing as a Process

Answer

In MediQueue, testing was performed only before release. Testing should instead be a continuous process throughout the software development life cycle.

1. Testing as a Process

Testing involves five important activities:

1. **Planning:** Decide testing objectives, resources and schedule.
2. **Design:** Prepare test cases and expected results.
3. **Execution:** Run test cases and identify defects.

4. **Evaluation:** Compare actual results with expected results.
5. **Reporting:** Record defects and communicate test results.

2. Verification and Validation

- **Verification:** Checks whether the product is being developed correctly according to specifications.
- **Validation:** Checks whether the final application meets user requirements.

Development phase	**Verification and validation activities**
Requirements	Review appointment rules and fee calculations
Design	Review token generation and patient-data handling
Coding	Perform unit testing and code reviews
Testing	Perform integration and system testing
Production	Monitor application performance and user complaints

3. Correct Phase for Detecting MediQueue Defects

Incident	**Defect**	**Phase where it should be detected**
I1	Duplicate token	Design and integration testing
I2	Sunday booking	Requirements review
I3	Incorrect consultation fee	Requirements and integration testing
I4	Application crash	Coding and unit testing
I5	Senior-citizen priority	Requirements review
I6	Wrong prescription details	Design and concurrency testing
I7	Incorrect user manual	Documentation review
I8	Wrong expected test result	Test design and review
I9	SMS reminder failure	Integration testing
I10	Slow schedule screen	Performance testing

4. Cost of Fixing Defects

The given assignment provides the following cost data:

Detection phase	Defects detected	Cost per defect	Total cost
Requirements	2	₹500	₹1,000
Design	3	₹1,500	₹4,500
Coding	15	₹3,000	₹45,000
Testing	20	₹8,000	₹1,60,000
Production	60	₹40,000	₹24,00,000
Total	**100**		**₹26,10,500**

Calculation:

Total cost = (2 × 500) + (3 × 1,500) + (15 × 3,000) + (20 × 8,000) + (60 × 40,000)

Total cost = ₹26,10,500.

The hospital spent ₹24,00,000 fixing defects after release alone. This shows the high cost of detecting defects late.

5. Expected Savings from Early Reviews

The table shows 30 defects introduced during requirements and 22 during design.

If all these 52 defects had been detected and corrected in their respective phases:

- Requirements: 30 × ₹500 = ₹15,000
- Design: 22 × ₹1,500 = ₹33,000

Total early correction cost = ₹48,000.

For an illustrative upper-bound comparison, if all 52 defects would otherwise have reached production:

- Production correction cost = 52 × ₹40,000 = ₹20,80,000.
- Early correction cost = ₹48,000.
- Potential saving = ₹20,80,000 − ₹48,000 = **₹20,32,000**.

This is an illustrative estimate, not an exact saving. The assignment does not provide a defect-by-defect link between the phase where a defect originated and the phase where it was detected. Therefore, the actual saving cannot be calculated exactly.

Conclusion

Testing throughout the life cycle helps identify defects early and reduces correction costs. MediQueue should conduct regular reviews, testing and defect reporting instead of depending only on final-stage testing.

TASK 4 – Apply and Critique Basic Definitions

Answer

Software testing terminology helps the team identify, understand and communicate defects correctly. Using terms carelessly can create confusion and affect software quality.

1. Basic Testing Definitions with MediQueue Examples

1. **Error:** A human mistake made during development. Example: Developers misunderstood the GST calculation.
2. **Fault (Defect):** A mistake in the software code, design or requirements. Example: Incorrect fee calculation logic in MediQueue.
3. **Failure:** The incorrect behaviour observed when the software runs. Example: The application displays ₹500 but charges ₹550.
4. **Test Case:** A set of inputs, conditions and expected results used to test a feature. Example: Testing whether a patient can book an appointment on Sunday.
5. **Test Oracle:** A source used to determine the correct expected result. Example: The approved hospital fee and GST rules are used to verify the consultation amount.
6. **Test Bed:** The environment and resources used to perform testing. Example: A testing setup containing the MediQueue application, database, devices and test data.

2. Error, Fault and Failure in Selected Incidents

Incident	Error	Fault	Failure
I3	GST rule misunderstood	Incorrect fee calculation logic	₹550 charged instead of the displayed ₹500
I5	Senior-citizen age rule misunderstood	Incorrect age eligibility condition	Priority not applied to a patient turning 60
I6	Incorrect handling of simultaneous saves	Shared patient data not properly isolated	Previous patient's details appear in another prescription

3. Why Did Incident I8 Occur?

In I8, the tester compared the refund output with an incorrect expected value and marked the test as passed.

This happened because the test oracle was wrong. Proper verification of expected results and independent review of test cases could have prevented this problem.

4. Need for a Common Glossary

MediQueue needs a common testing glossary so that developers and testers understand and use the same terminology.

It should contain:

- Error, fault and failure.
- Test case and test oracle.
- Test bed and test environment.
- Defect severity and priority.
- Testing status and defect status.

Conclusion

Correct testing terminology improves communication, defect reporting and testing accuracy. A common glossary would help MediQueue avoid confusion and improve software quality.

TASK 5 – Evaluate the Application of Testing Principles

Answer

Software testing principles are basic guidelines that help identify defects effectively. The developers' claim that testing everything proves the software is correct is wrong because testing cannot guarantee that software is completely defect-free.

Important Software Testing Principles

1. Testing Shows the Presence of Defects

Testing can identify defects but cannot prove that no defects exist.

Example: MediQueue passed its demonstration, but patients later reported duplicate tokens and incorrect fees.

2. Exhaustive Testing Is Impossible

It is not possible to test every input and condition.

Example: MediQueue has thousands of daily users. All possible appointment times, patient names and simultaneous bookings cannot be tested manually.

3. Early Testing Saves Time and Cost

Defects should be identified during requirements and design stages.

Example: I2 could have been prevented by reviewing the appointment requirements before development.

4. Defect Clustering

A small number of modules usually contain most of the defects.

Example: MediQueue has several problems related to appointment booking, payment and patient-data handling. These modules need more testing attention.

5. Pesticide Paradox

Repeating the same test cases may not identify new defects. Test cases should be regularly updated.

Example: MediQueue should include new test cases for simultaneous bookings and long patient names.

6. Testing Is Context Dependent

Testing methods should depend on the type of application.

Example: Since MediQueue is a healthcare application, it needs strong security, privacy, accuracy and reliability testing.

7. Absence-of-Errors Fallacy

Even if software has very few defects, it may still fail to meet user requirements.

Example: The application may work without crashing but still accept appointments on Sundays when the hospital is closed.

Conclusion

The most important principle for MediQueue is early testing because it helps identify defects before release and reduces correction costs. The team should follow all testing principles to improve application quality and patient safety.

TASK 6 – Evaluate the Tester's Role

Answer

A tester plays an important role in identifying defects, checking requirements and ensuring that software meets user expectations.

MediQueue had no independent tester, which allowed several defects to reach patients.

1. Role and Responsibilities of a Tester

- Understand and review software requirements.
- Prepare and execute test cases.
- Identify and report defects.
- Verify fixes and perform regression testing.
- Check performance, security and reliability.
- Prepare test reports before release.

2. Developers Testing Their Own Code

Advantages:

- Developers understand their code.
- Defects can be identified quickly.
- Testing can be performed during development.

Limitations:

- Developers may overlook their own mistakes.
- They may test only expected situations.
- They may have bias towards their implementation.
- Independent verification may be missing.

3. Incidents an Independent Tester Could Have Detected

Incident	**Testing activity**
I1	Simultaneous booking testing
I2	Requirement validation
I3	Payment and fee verification
I4	Boundary value testing
I5	Age eligibility testing
I6	Concurrent prescription testing
I7	Documentation testing
I8	Test case and expected-result review
I9	API integration testing
I10	Performance testing

4. Qualities of a Healthcare Application Tester

A tester should have:

- Technical knowledge of software testing.
- Attention to detail.
- Analytical and problem-solving skills.
- Knowledge of patient privacy and data security.
- Good communication skills.
- Responsibility and ethical behaviour.

Conclusion

An independent tester would have significantly improved MediQueue's quality by identifying defects that developers overlooked. However, a tester alone cannot prevent every defect. Testing must be supported by proper requirements, reviews and development processes.

TASK 7 – Trace the Origins of Defects

Answer

Defect origin analysis helps identify the development phase where a defect was introduced. It helps the organization understand the root causes and prevent similar problems in the future.

1. Origin of Each MediQueue Incident

Incident	Defect origin	Evidence and reason
I1	Design/Coding	Simultaneous bookings generate duplicate tokens due to improper concurrency handling.
I2	Requirements	Sunday closure rules were not included in the requirements.
I3	Requirements/Coding	GST rules were misunderstood, resulting in incorrect fee calculation.
I4	Coding	The application cannot handle patient names

		exceeding 40 characters.
I5	Requirements/Coding	The age eligibility rule does not correctly handle patients turning 60 on the appointment date.
I6	Design/Coding	Patient data is not properly isolated when two doctors save prescriptions simultaneously.
I7	Documentation	The manual mentions a cancellation menu that does not exist.
I8	Testing	An incorrect expected value was used to mark a refund test as passed.
I9	Environment/Integration	The SMS integration does not handle the provider's API key change correctly.
I10	Design/Coding	The schedule screen has poor performance on the hospital network.

2. Defects with Multiple Origins

Some incidents may have more than one origin.

- I1 involves design and coding because the system does not properly manage simultaneous requests.
- I3 involves requirements and coding because the GST rule was misunderstood and implemented incorrectly.
- I6 involves design and coding because patient-data isolation was not properly handled.
- I9 involves integration and environment because the external SMS provider changed its API key.

3. Overall Pattern of Defect Origins

The incidents show that many defects originated from unclear requirements, poor design, coding mistakes and insufficient testing.

The hospital should invest more effort in:

- Requirement reviews.
- Design inspections.

- Coding standards.
- Independent testing.
- Integration and performance testing.
- Proper documentation.

Conclusion

Most MediQueue defects could have been identified earlier through proper reviews and testing. The hospital should focus on preventing defects at their origin rather than correcting them after patients report problems.

TASK 8 – Evaluate Defect Classes and Severity

Answer

Defect classification helps identify the nature and impact of software problems. It allows the hospital to decide which defects must be corrected first.

1. Types of Defect Classes

1. **Requirements defects:** Missing or incorrect requirements.
2. **Design defects:** Problems in system architecture or logic.
3. **Coding defects:** Errors in program implementation.
4. **Testing defects:** Incorrect test cases or expected results.
5. **Documentation defects:** Incorrect or incomplete user instructions.
6. **Integration and environment defects:** Problems caused by external systems or configurations.

2. Classification of MediQueue Incidents

Severity indicates the impact of a defect, while priority indicates how urgently it should be fixed.

ID	Defect class	Severity	Priority
I1	Design/Coding	High	High
I2	Requirements	Medium	High
I3	Requirements/Coding	High	High
I4	Coding	High	High
I5	Requirements/Coding	High	High
I6	Design/Coding	Critical	Critical
I7	Documentation	Low	Medium
I8	Testing	High	High
I9	Integration/Environment	Medium	High
I10	Design/Coding	Medium	Medium

3. Explanation of Severity and Priority

- **I1 – Duplicate Token:** High severity because two patients receive the same token, creating appointment confusion. High priority because it must be corrected quickly.
- **I2 – Sunday Booking:** Medium severity because appointments are accepted on a closed day. High priority because it affects appointment scheduling.
- **I3 – Wrong Fee:** High severity because patients are charged an incorrect amount.
- **I4 – Application Crash:** High severity because users cannot complete certain operations.
- **I5 – Senior-Citizen Priority:** High severity because eligible patients do not receive priority.
- **I6 – Wrong Prescription:** Critical severity because confidential medical information is displayed incorrectly.
- **I7 – Incorrect Manual:** Low severity because it provides incorrect instructions, but cancellation access still needs correction.
- **I8 – Wrong Test Result:** High severity because incorrect testing decisions allow defects to reach production.

- **I9 – SMS Failure:** Medium severity because patients may miss appointment reminders.
- **I10 – Slow Screen:** Medium severity because it affects application performance and user experience.

4. Difference Between Severity and Priority

Severity measures how seriously a defect affects the system, while priority determines how urgently it should be fixed.

For example, I1 may occur only during simultaneous bookings, but it has high severity because it creates duplicate tokens. I7 has lower severity but may still need attention because patients cannot follow the cancellation instructions.

5. Defect Class to Target First

The hospital should first target requirements and design defects because they can create several problems in later development stages.

It should also give immediate attention to critical patient-data defects such as I6.

Conclusion

Defect classification helps MediQueue identify high-risk problems and allocate resources effectively. Patient safety, privacy and financial accuracy should receive the highest attention.

TASK 9 – Design and Appraise a Defect Repository

Answer

A defect repository is a centralized system used to record, track, manage and analyze software defects throughout the development life cycle.

Currently, MediQueue records complaints through emails and spreadsheets, which makes defect tracking difficult.

1. Important Fields in a Defect Repository

Field	Purpose
Defect ID	Gives a unique identification number
Defect Title	Provides a short description
Description	Explains the problem clearly
Module	Identifies the affected application module
Steps to Reproduce	Helps reproduce the defect
Expected Result	Describes correct behaviour
Actual Result	Records the observed behaviour
Severity	Identifies the impact of the defect
Priority	Defines how urgently it must be fixed
Reported By	Identifies who reported the defect
Assigned To	Identifies the responsible developer
Status	Tracks whether the defect is open, fixed or closed
Root Cause	Identifies why the defect occurred
Test Evidence	Stores screenshots, logs and test results
Resolution Date	Records when the defect was fixed

2. Benefits of a Defect Repository

- Helps track defects from identification to closure.
- Prevents duplicate defect reports.
- Helps identify frequently occurring problems.
- Supports root-cause analysis.
- Helps management monitor testing progress.
- Supports future process improvement.

3. Weaknesses of the Current Approach

Emails and spreadsheets may result in missing information, duplicate complaints and difficulty tracking defect status. They also make it difficult to identify repeated problems and analyze defect trends.

Conclusion

MediQueue should implement a centralized defect repository to maintain accurate records, improve communication and prevent repeated defects. It will help the team improve software quality continuously.

TASK 10 – Evaluate Defect Examples and Prevention Strategies

Answer

Defect prevention means identifying and removing the root causes of defects before they reach users. MediQueue can prevent several problems through simple changes in its development and testing process.

1. Three Defects and Their Prevention

Defect 1 – I2: Sunday Appointment Booking

- **Defect Class:** Requirements defect.
- **Root Cause:** The Sunday hospital closure rule was not included in the requirements.
- **Prevention Strategy:** Conduct requirement reviews with hospital management before development.
- **Cost and Benefit:** Reviews require little time and effort but prevent incorrect appointment rules from being implemented.

Defect 2 – I1: Duplicate Token Numbers

- **Defect Class:** Design/Coding defect.
- **Root Cause:** The system does not properly handle simultaneous appointment requests.
- **Prevention Strategy:** Conduct design inspections and concurrency testing.

- **Cost and Benefit:** Design reviews and testing require additional effort but prevent booking conflicts and improve reliability.

Defect 3 – I8: Incorrect Refund Test Result

- **Defect Class:** Testing defect.
- **Root Cause:** The tester used an incorrect expected value.
- **Prevention Strategy:** Introduce test case reviews and verify expected results against approved business rules.
- **Cost and Benefit:** Reviewing test cases requires little effort and helps prevent incorrect test approvals.

2. Best Value Strategy

Requirement reviews provide the best value because they identify missing business rules before coding begins. The given cost data shows that fixing a defect during requirements costs only ₹500, compared with ₹40,000 in production.

Conclusion

MediQueue should introduce requirement reviews, design inspections and test case reviews. These activities require relatively little effort but can prevent expensive defects and improve application quality.

TASK 11 – Recommend and Defend a Quality Improvement Plan

Answer

MediQueue needs a structured quality improvement plan to reduce defects, improve reliability and protect patient information.

1. Recommended Process Changes

Step 1 – Requirement Review

Review all requirements with hospital management and identify missing rules such as Sunday holidays, GST and senior-citizen eligibility.

Step 2 – Design Review

Inspect the system design to identify problems related to duplicate tokens, simultaneous requests and patient-data handling.

Step 3 – Coding Standards

Follow coding standards, input validation and secure programming practices during development.

Step 4 – Independent Testing

Appoint a tester to prepare and execute test cases, identify defects and verify corrections.

Step 5 – Different Levels of Testing

Perform unit testing, integration testing, system testing, acceptance testing, performance testing and security testing.

Step 6 – Defect Repository

Maintain a centralized defect repository to record defects, root causes, severity, priority and resolution status.

Step 7 – Continuous Monitoring

Monitor application performance, user complaints and production defects after release.

2. Role of Testers and Reviews

Testers should participate throughout the development life cycle. Requirement and design reviews should identify defects early. The defect repository should maintain proper records and support future improvement.

3. Expected Effect on Defects and Costs

The plan will help detect defects during requirements, design and coding instead of production.

The given data shows that production defects cost ₹40,000 each, whereas requirement defects cost only ₹500 each. Therefore, early defect detection can significantly reduce correction costs.

4. Trade-offs and Limitations

- Testing and reviews require additional time.
- Hiring an independent tester increases project cost.
- Maintaining a defect repository requires regular updates.
- A small team may have limited resources.

However, these activities reduce the risk of serious defects and expensive production failures.

5. Practicality for a Small Team

The plan is practical for five developers. Developers can share review responsibilities, use automated testing tools and maintain a simple defect tracking system. One team member can coordinate testing activities without requiring a large testing department.

Conclusion

MediQueue should adopt a structured quality improvement process involving reviews, independent testing, defect tracking and continuous monitoring. This will reduce defects, improve patient safety and increase software reliability.

TASK 12 – Re-evaluate and Adapt the Quality Process

Answer

The hospital has introduced two major changes to MediQueue. It now wants to release software updates every two weeks instead of every six months. It must also connect to the national digital health ID system and share patient health records between hospitals.

These changes increase the need for continuous testing, security and data accuracy.

A. Impact Analysis

1. Changes from the Original Situation

- Release frequency changes from six months to two weeks.
- The application connects to an external national health ID system.
- Patient health records are shared between hospitals.
- Security, privacy and integration risks increase.

2. Assumptions Affected

The original plan assumed sufficient time for manual testing before each release. With two-week releases, manual testing alone may not be sufficient.

3. Effect of Frequent Releases

Frequent releases require continuous testing, automated regression testing and faster defect reporting. Testers must participate throughout development rather than only before release.

4. Increased Patient-Safety Risks

Sharing medical records increases the seriousness of defects related to patient identity, privacy, security and data accuracy. Incorrect patient information could affect medical decisions.

B. Evaluation of the Original Plan

1. Suitability of Reviews and Testing

The original reviews and testing activities are useful but must be completed within every two-week development cycle. Automation should be introduced to save time.

2. New Defect Origins

The external health ID integration may introduce:

- API communication defects.
- Authentication and authorization defects.
- Data format and compatibility defects.
- Network and connectivity failures.
- Patient identity mismatches.
- Data privacy and security defects.

3. Defect Repository

The existing defect repository is useful but needs additional fields such as external system ID, API version, integration status, security impact and data privacy impact.

C. Adaptation or Redesign

Decision: Modify the Original Quality Improvement Plan.

A completely new process is not necessary because the original plan already includes reviews, testing and defect tracking. However, it must be improved to support frequent releases and external integration.

1. Revised Quality Process

1. Review requirements before every development cycle.
2. Conduct design and security reviews.
3. Perform unit and integration testing continuously.
4. Automate regression testing.
5. Perform API and compatibility testing with the national health ID system.
6. Conduct security and privacy testing.
7. Maintain a centralized defect repository.
8. Monitor the application after every release.

2. Revised Role of the Tester

The tester should work with developers throughout each two-week cycle. The tester must validate new features, perform regression testing, verify external API integration and check patient-data security.

3. Important Testing Principles

- Early testing to detect defects before development is completed.
- Exhaustive testing is impossible, so high-risk areas must receive more attention.
- Testing should be context dependent, especially for healthcare applications.
- Regression testing is important after every update.
- Defect prevention is better than correcting defects after release.

4. Tracking External Defects

External system defects should be recorded separately in the defect repository with their API details, error logs, affected records, severity, priority and responsible organization. This will help the hospital coordinate with the external agency.

D. Evidence-Based Defence

The decision to modify the original plan is supported by the following evidence:

1. Defect Cost Data

The assignment shows that production defects cost ₹40,000 each, compared with ₹500 during requirements. Therefore, early testing and automation can reduce expensive production defects.

2. Defect Origins

MediQueue experienced defects in requirements, design, coding, testing and integration. The revised process addresses these problems through reviews and continuous testing.

3. Frequent Releases

Two-week releases require faster testing and automated regression checks to ensure that new changes do not affect existing features.

4. Patient Safety and Data Privacy

Sharing medical records between hospitals increases the importance of secure authentication, correct patient identification and data protection.

5. Team Size

The hospital has only five developers. Automated testing tools, shared review responsibilities and a centralized defect repository will make the process manageable without requiring a large testing team.

Final Conclusion

MediQueue should modify its original quality improvement plan rather than retain it completely or redesign it from the beginning. The revised process should include continuous testing, automated regression testing, API integration testing, security testing and improved defect tracking.

This approach is practical for a small development team and will improve software reliability, protect patient information and reduce the cost of defects in future releases.